DO NOW — RECALL

Programming paradigms: applied comparison

Name: _____ Class: _____ Date: _____

1. How does OOP differ from a procedural program?

2. What does a local variable mean?

3. If a field is private-set, how can callers change it safely?

4. Why does `Deposit(balance,5);` not update the caller's int balance if only a return value is produced?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

OOP groups state and behaviour into objects; procedural code is organised around procedures/functions.

2. Model answer

A variable accessible only within its declaration's scope.

3. Model answer

Through public methods/property logic defined by the class.

4. Model answer

The return is discarded and a value-type parameter is a copy; the caller must assign the returned result.

Vocabulary — know these terms

refactoring — Restructuring code while preserving expected behaviour.

encapsulation — Controlled access to internal state.

instance — Individual object created from a class.

return value — Output returned to the caller.

side effect — Observable change beyond a returned result.

NEW LEARNING

Read, annotate and explain

Procedural state

The caller passes an integer balance and explicitly stores the returned value.

Object state

Account object A stores Balance internally and Deposit changes that object's state.

Separate instances

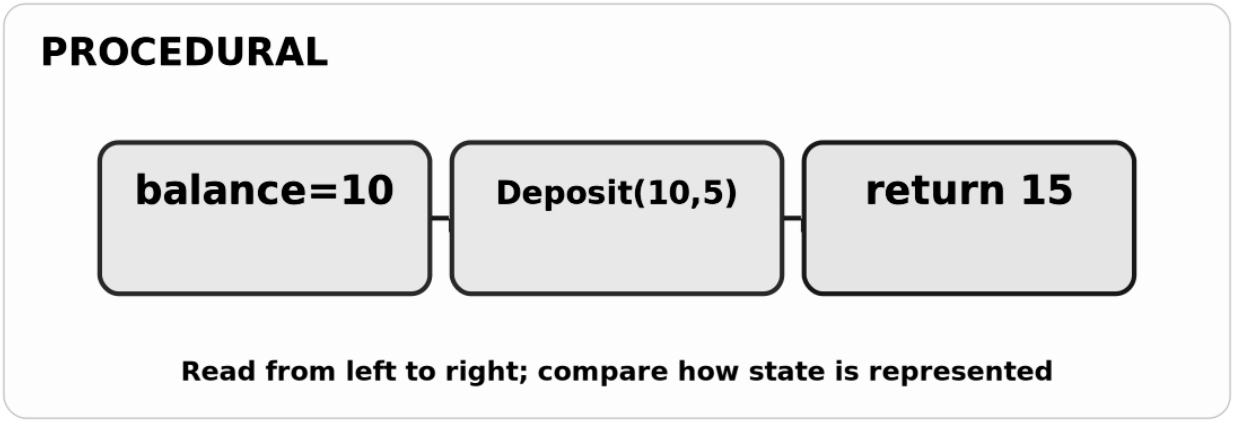
Account B stays at 20 when Account A changes to 15; the objects have independent state.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

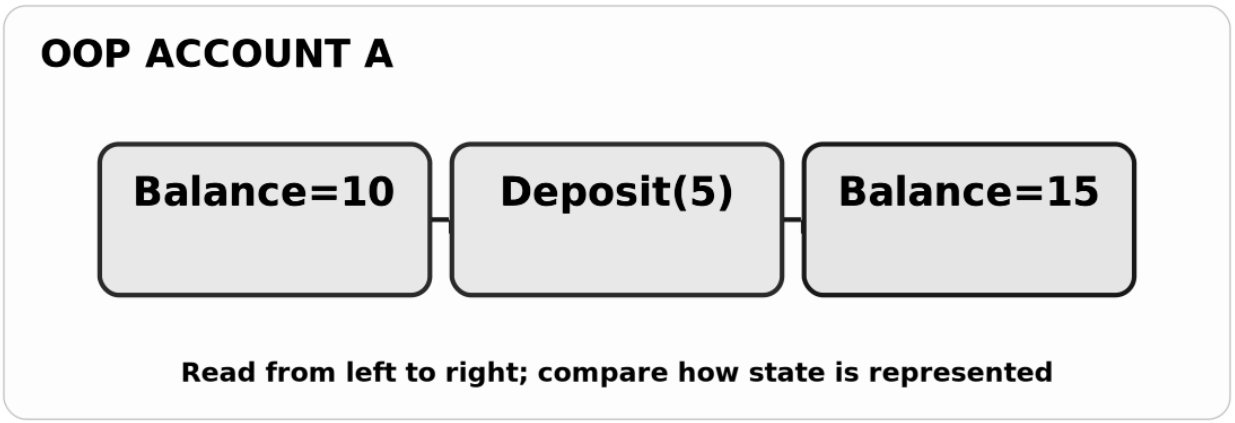
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



PROCEDURAL; balance=10; Deposit(10,5); return 15

New Learning

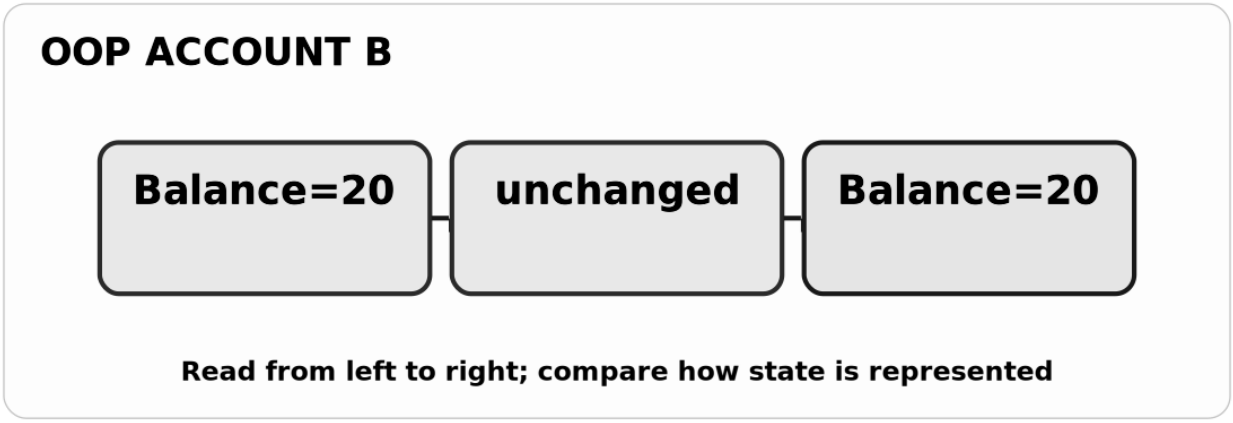


OOP ACCOUNT A; Balance=10; Deposit(5); Balance=15

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



OOP ACCOUNT B; Balance=20; unchanged; Balance=20

I DO — WORKED EXAMPLE

Follow the worked method

Two accounts start at 8 and 20; only first account deposits 4. Trace both.

Predict

Account a's Balance becomes 12.

Model response

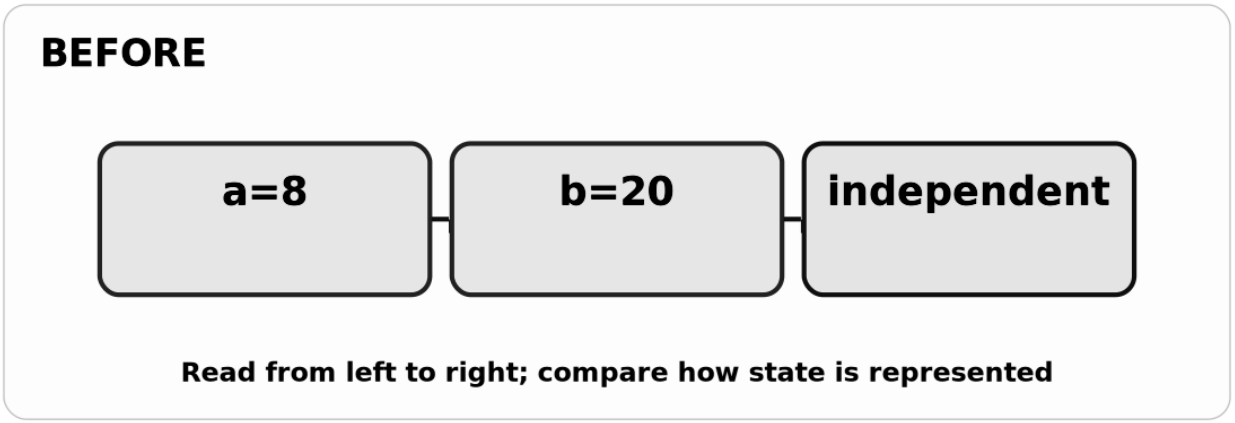
Account b remains at 20 because its state is separate.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Two accounts start at 8 and 20; only first account deposits 4. Trace both.

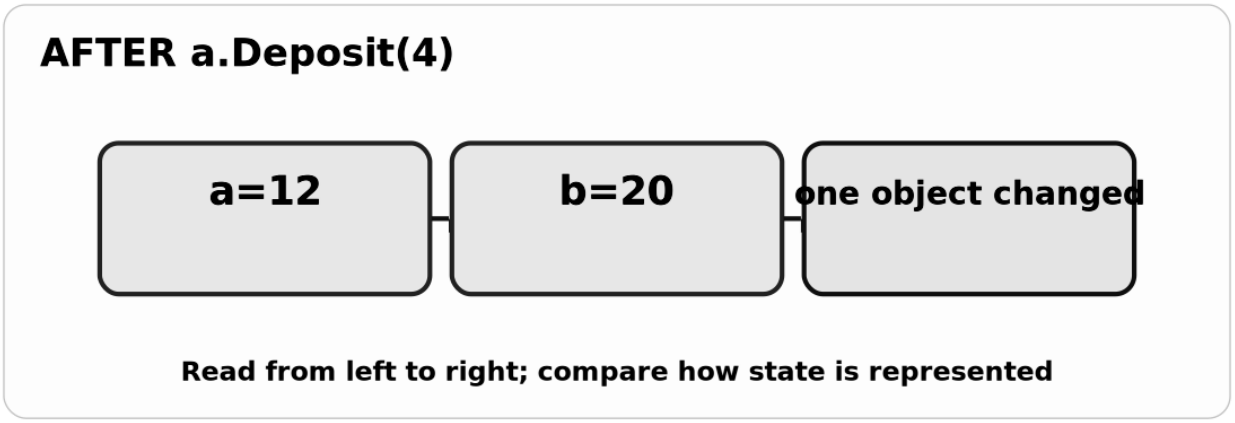
Account a's Balance becomes 12.



BEFORE; a=8; b=20; independent

Two accounts start at 8 and 20; only first account deposits 4. Trace both.

Account a's Balance becomes 12.



AFTER a.Deposit(4); a=12; b=20; one object changed

WE DO — GUIDED PRACTICE

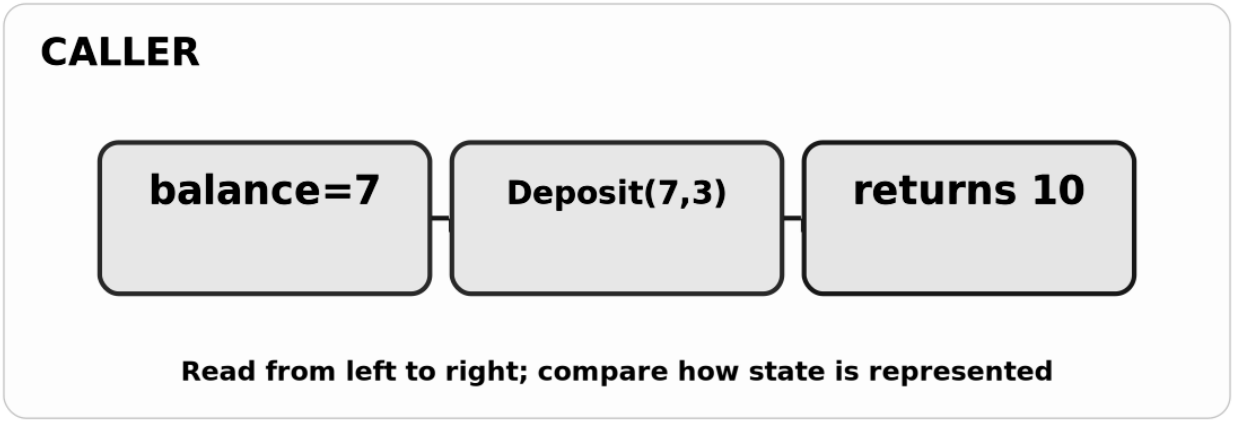
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A procedural function returns balance+3, but the caller ignores the return. What happens?

Deposit computes and returns 10, but its result is ignored.



CALLER; balance=7; Deposit(7,3); returns 10

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict Deposit computes and returns 10, but its result is ignored.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Procedural AddFive(x) returns x+5. If caller x=2 discards the result, what is x?

Question 2

Two objects have balances A=10, B=30. A.Deposit(7) and B.Deposit(2) run. Find both balances and describe why they differ.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Compare one maintenance advantage and one risk of procedural and OOP implementations for a library system.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

Calling a C# function that returns a new balance automatically changes the caller's original int variable.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

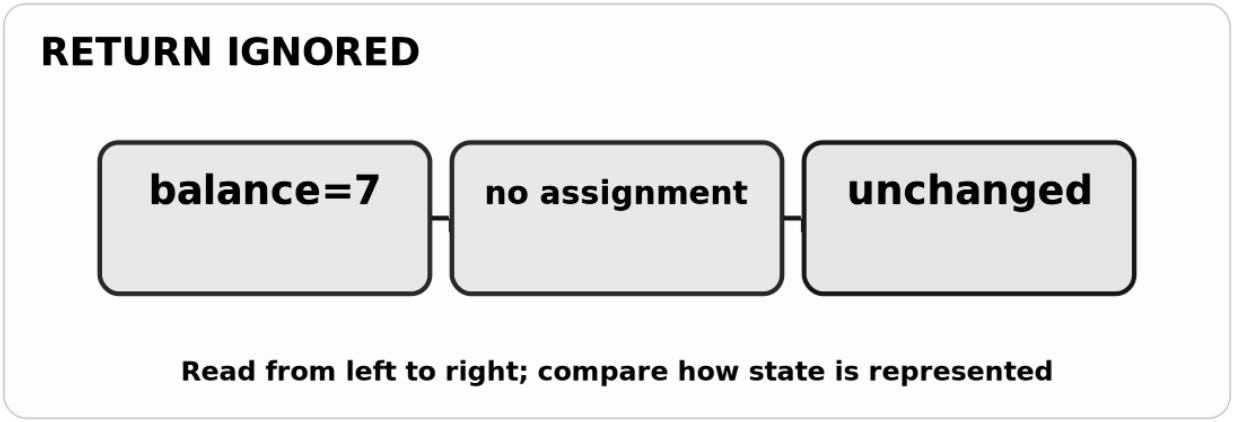
Next step: revisit the model or a relevant revision-guide section if needed.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A procedural function returns balance+3, but the caller ignores the return. What happens?

Deposit computes and returns 10, but its result is ignored.



RETURN IGNORED; balance=7; no assignment; unchanged

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Model response

The caller's balance remains 7. Assign balance=Deposit(balance,3) to store 10.

IMPROVE: Explain one correction to your own reasoning.